Sabri Rebaine 12210775
Arthur Schorter 12202370
INFO 2

Le 30/03/2026

Rapport TP5 : Base de données

Discipline : Base de données avancées
Enseignant : Monsieur Youcef Samir

Exercice 1

1.

On fait sur les 2 terminaux :

```
Entrez le nom utilisateur : INFO2_13
Entrez le mot de passe : ********
Entrez le mot de passe :
Heure de la derni□re connexion r□ussie : Sam. Nov.  01 2025 03:51:56 +02:00

Connect□□:
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production

SQL> SET AUTOCOMMIT OFF
SQL> CREATE TABLE transaction (
  2  idTransaction VARCHAR2 (44) ,
  3  valTransaction NUMBER (10) ) ;

Table cr□□e.
```

2.

On fait sur le premier terminal :

```
SQL> CREATE TABLE transaction (
  2  idTransaction VARCHAR2 (44) ,
  3  valTransaction NUMBER (10) ) ;

Table cr□□e.
```

On insère ensuite des transactions :

```
SQL> INSERT INTO transaction VALUES ('T001', 100);

1 ligne cr□□e.

SQL> INSERT INTO transaction VALUES ('T002', 200);

1 ligne cr□□e.

SQL> INSERT INTO transaction VALUES ('T003', 300);

1 ligne cr□□e.
```

On mets à jours une ligne et on en supprime une :

```
SQL> UPDATE transaction SET valTransaction = 999 WHERE idTransaction = 'T001';

1 ligne mise ►jour.

SQL> DELETE FROM transaction WHERE idTransaction = 'T003';

1 ligne supprim►e.
```

On voit que la ligne T001 a bien été modifiée et que la ligne T003 a bien été supprimée :

```
SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T001                                                    999
T002                                                    200

SQL>
```

On annule enfin ce que l'on vient de faire avec un ROLLBACK :

```
SQL> ROLLBACK
  2  ;

Annulation (rollback) effectu►e.

SQL> SELECT * FROM transaction;

aucune ligne s►lectionn►e

SQL>
```

Les lignes que l'on avait créées et tout ce qu'on avait effectuées ne sont plus présentes.

3.

Depuis la 2ème session, on réinsère des lignes dans la table transaction :

```
SQL> INSERT INTO transaction VALUES ('T010', 10);

1 ligne cr??e.

SQL> INSERT INTO transaction VALUES ('T011', 20);

1 ligne cr??e.
```

On quitte ensuite brutalement avec un quit. On regarde sur la session 1 si les changements ont été pris en compte :

```
SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T010                                                     10
T011                                                     20
```

On voit que les ajouts ont bien été pris en compte, la commande quit a donc fait un Commit explicite.

4.

Sur la session 1, on ajoute de nouvelles lignes dans la table Transaction, on ajoute les transactions T020 et T021, on ferme ensuite le terminal avec alt+f4 et on se reconnecte. On regarde si les précédentes modifications ont été prises en compte :

```
SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T010                                                     10
T011                                                     20
```

Les lignes T020 et T021 ne sont plus dispoibles. Si on ferme le terminal sans quit ou commit, un Rollback est effectué automatiquement.

5.

On se reconnecte à la session avec le terminal 2. On insère de nouvelles lignes dans la table transaction :

```
SQL> INSERT INTO transaction VALUES ('T030', 111);

1 ligne cr??e.

SQL> INSERT INTO transaction VALUES ('T031', 222);

1 ligne cr??e.
```

On modifie ensuite la structure de la table :

```
SQL> ALTER TABLE transaction ADD val2transaction NUMBER(10);

Table modifi?e.
```

On effectue ensuite un ROLLBACK et on regarde l'état de la table :

```
SQL> ROLLBACK;

Annulation (rollback) effectu?e.

SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION VAL2TRANSACTION
-------------------------------------------- -------------- ---------------
T030                                                    111
T031                                                    222
T010                                                     10
T011                                                     20
```

Malgré l'annonce ROLLBACK effectuée, les lignes ajoutées sont toujours présentes.

Conclusion ex1

Une session correspond à une connexion d'un utilisateur à la base de données. Elle débute à la connexion et se termine à la déconnexion. Une même session peut contenir plusieurs transactions exécutées en série.

Une transaction est une suite d'opérations SQL formant une unité logique indivisible. Elle commence implicitement après le démarrage de la session ou après la fin de la transaction précédente (COMMIT ou ROLLBACK). On la valide avec COMMIT, ce qui rend les modifications permanentes et visibles par les autres sessions, ou on l'annule avec ROLLBACK, ce qui remet la base dans l'état qu'elle avait au début de la transaction.

Les manipulations réalisées ont mis en évidence plusieurs comportements importants. D'abord, un ROLLBACK annule bien toutes les modifications effectuées depuis le dernier COMMIT, la table retrouve son état initial. Ensuite, quitter la session avec la commande quit provoque un COMMIT implicite : les données insérées sans COMMIT explicite sont tout de même sauvegardées. En revanche, une fermeture brutale de sqlplus (par exemple via ctrl+c) provoque un ROLLBACK implicite, et les données non validées sont perdues. Enfin, l'exécution d'une commande DDL comme ALTER TABLE déclenche également un COMMIT implicite avant son exécution : toutes les modifications en attente sont alors validées définitivement, et le ROLLBACK exécuté ensuite n'a aucun effet sur elles.

En résumé, une transaction respecte le principe d'atomicité : soit toutes ses opérations sont validées ensemble, soit elles sont toutes annulées. La validation est définitive et irréversible (principe de durabilité), tandis que les modifications non validées restent invisibles pour les autres sessions (principe d'isolation).

Exercice 2

On créer les tables vols et clients comme données dans le TP :

```
SQL> CREATE TABLE vol (
  2      idVol               VARCHAR2(44),
  3      capaciteVol         NUMBER(10),
  4      nbrPlacesReserveesVol NUMBER(10)
  5  );

Table cr??.

SQL> CREATE TABLE client (
  2      idClient                VARCHAR2(44),
  3      prenomClient            VARCHAR2(11),
  4      nbrPlacesReserveesCleint NUMBER(10)
  5  );

Table cr??.
```

On ajoute ensuite un vol et deux clients qu’on valide par un COMMIT :

```
SQL> INSERT INTO vol VALUES ('V001', 10, 0);

1 ligne cr??e.

SQL> INSERT INTO client VALUES ('C001', 'Alice', 0);

1 ligne cr??e.

SQL> INSERT INTO client VALUES ('C002', 'Bob',   0);

1 ligne cr??e.

SQL> COMMIT;

Validation effectu?e.
```

Puis on vérifie l’état de nos deux tables :

```
SQL> SELECT * FROM vol;
SELECT * FROM client;

IDVOL                                          CAPACITEVOL NBRPLACESRESERVEESVOL
---------------------------------------------- ----------- ---------------------
V001                                                    10                     0

SQL>
IDCLIENT                                       PRENOMCLIEN
---------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C001                                           Alice
                      0

C002                                           Bob
                      0
```

Isolation des transactions (READ COMMITTED)

Sur le terminal 1, on effectue une réservation de 2 billets pour Alice sans faire de commit et on regarde l’état des tables du côté du terminal 1 :

```
SQL> SELECT * FROM vol;

IDVOL                                      CAPACITEVOL NBRPLACESRESERVEESVOL
------------------------------------------ ----------- ---------------------
V001                                                10                     2

SQL> SELECT * FROM client;

IDCLIENT                                   PRENOMCLIEN
------------------------------------------ -----------
NBRPLACESRESERVEESCLEINT
------------------------
C001                                       Alice
                       2

C002                                       Bob
                       0
```

On regarde l’état des mêmes tables du côté du terminal 2 :

```
SQL> SELECT * FROM vol;

IDVOL                                      CAPACITEVOL NBRPLACESRESERVEESVOL
------------------------------------------ ----------- ---------------------
V001                                                10                     0

SQL> SELECT * FROM client;

IDCLIENT                                   PRENOMCLIEN
------------------------------------------ -----------
NBRPLACESRESERVEESCLEINT
------------------------
C001                                       Alice
                       0

C002                                       Bob
                       0
```

Les modifications n’ont pas été prises en compte car on a pas effectuer de COMMIT sur le terminal 1.

ROLLBACK de T1

On effectue un Rollback sur le terminal 1.

Les deux session voient maintenant la même chose, T2 n’a jamais vu les modifications de T1 :

```
SQL> SELECT * FROM client;

IDCLIENT                                           PRENOMCLIEN
-------------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C001                                               Alice
                       0

C002                                               Bob
                       0


SQL> SELECT * FROM vol;

IDVOL                                              CAPACITEVOL NBRPLACESRESERVEESVOL
-------------------------------------------------- ----------- ---------------------
V001                                                        10                     0
```

COMMIT de T1 puis observation par T2

On refait la réservation de 2 billets pour Alice sur T1 et on commit :

```
SQL> UPDATE vol    SET nbrPlacesReserveesVol      = nbrPlacesReserveesVol
 2 WHERE idVol    = 'V001';

1 ligne mise ►jour.

SQL> UPDATE client SET nbrPlacesReserveesCleint   = nbrPlacesReserveesCleint
 2 WHERE idClient = 'C001';

1 ligne mise ►jour.

SQL> COMMIT;

Validation effectu►e.
```

On tente un Rollback depuis T1 qui dit être accepté mais, comme vu dans la partie 1, un Rollback après un Commit n’est pas pris en compte. Les modifications effectuées sont donc toujours visible sur T1.

On regarde maintenant si ces modifications sont visibles sur T2 :

```
SQL> SELECT * FROM vol;

IDVOL                                      CAPACITEVOL NBRPLACESRESERVEESVOL
------------------------------------------ ----------- ---------------------
V001                                                10                     2

SQL> SELECT * FROM client;

IDCLIENT                                   PRENOMCLIEN
------------------------------------------ -----------
NBRPLACESRESERVEESCLEINT
------------------------
C001                                       Alice
                       2

C002                                       Bob
                       0
```

Les modifications sont également visibles sur T2, dû au commit.

Mise à jour perdue (Lost Update) en READ COMMITTED

On revient à l'état initial des bases depuis T1 :

```
SQL> UPDATE vol    SET nbrPlacesReserveesVol      = 0 WHERE idVol    = 'V001';

1 ligne mise ►jour.

SQL> UPDATE client SET nbrPlacesReserveesCleint   = 0 WHERE idClient = 'C001';

1 ligne mise ►jour.

SQL> UPDATE client SET nbrPlacesReserveesCleint   = 0 WHERE idClient = 'C002';

1 ligne mise ►jour.

SQL> COMMIT;

Validation effectu►e.

SQL>
```

On réalise maintenant des transactions entrelacées comme suit :

Sur T1, on lit le vol et le client Alice :

```
SQL> SELECT nbrPlacesReserveesVol FROM vol    WHERE idVol    = 'V001';

NBRPLACESRESERVEESVOL
---------------------
                    0

SQL> SELECT nbrPlacesReserveesCleint FROM client WHERE idClient = 'C001';

NBRPLACESRESERVEESCLEINT
------------------------
                       0
```

Sur T2, on lit le vol et le client Bob :

```
SQL> SELECT nbrPlacesReserveesVol FROM vol    WHERE idVol    = 'V001';

NBRPLACESRESERVEESVOL
---------------------
                    0

SQL> SELECT nbrPlacesReserveesCleint FROM client WHERE idClient = 'C002';

NBRPLACESRESERVEESCLEINT
------------------------
                       0
```

Sur T1, on réserve 2 billets pour Alice et on COMMIT. Sut T2, on réserve 3 billets pour Bob et on COMMIT :

sur T1 :

```
SQL> UPDATE vol    SET nbrPlacesReserveesVol      = 0 + 2 WHERE idVol    = 'V00UPDATE vol    SET nbrPlacesReserveesVol      = 0 + 2 WHERE idVol    = 'V001';

1 ligne mise ►jour.

SQL> UPDATE client SET nbrPlacesReserveesCleint   = 0 + 2 WHERE idClient = 'C001';

1 ligne mise ►jour.

SQL> COMMIT;
     COMMIT;

Validation effectu►e.
```

Sur T2 :

```
SQL> UPDATE vol    SET nbrPlacesReserveesVol      = 0 + 3 WHERE idVol    = 'V001
';   UPDATE vol    SET nbrPlacesReserveesVol      = 0 + 3 WHERE idVol    = 'V001
';

1 ligne mise ►jour.

SQL> UPDATE client SET nbrPlacesReserveesCleint   = 0 + 3 WHERE idClient = 'C002
';

1 ligne mise ►jour.

SQL> COMMIT;

Validation effectu►e.
```

On vérifie les résultats et on a bien sur chacun des terminaux :

```
SQL> SELECT * FROM vol;
SELECSELECT * FROM vol;

IDVOL                                          CAPACITEVOL NBRPLACESRESERVEESVOL
---------------------------------------------- ----------- ---------------------
V001                                                    10                     3

SQL> SELECT * FROM client;

IDCLIENT                                       PRENOMCLIEN
---------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C001                                           Alice
                       2

C002                                           Bob
                       3
```

On a ici une incohérence, la base reflète 3 réservations sur le vol mais 5 côté clients.

Conclusion ex2

Oracle n'implémente PAS le verrouillage à deux phases (2PL) classique vu en cours. Il utilise la MVCC (Multi-Version Concurrency Control) :

- En READ COMMITTED : chaque lecture voit la dernière version commitée, sans poser de verrou de lecture. Les écritures posent un verrou exclusif jusqu'au COMMIT/ROLLBACK.
  => Problème de mise à jour perdue possible.

- En SERIALIZABLE : Oracle détecte qu'une ligne lue en début de transaction

a été modifiée et commitée par une autre transaction. Il rejette alors la transaction courante avec ORA-08177. Ce n'est pas du 2PL (pas de verrou de lecture bloquant), mais le résultat est équivalent : les exécutions concurrentes sont sérialisables.

En résumé : Oracle garantit la sérialisabilité sans blocage par verrouillage en lecture grâce à MVCC + détection de conflits à la validation.